République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supèrieur et de la Recherche Scientifique
USTHB
Département d'Informatique
Professeur H. NACER

Support de cours

Sécurité

Année Universitaire : 2025/ 2026

Chapitre 3: Cryptographie symétrique

Auparavant, la sécurité reposait sur le fait que l'algorithme utilisé était secret, par exemple : l'alphabet de César : décalage de trois positions des lettres de l'alphabet. Aujourd'hui : les algorithmes sont connus de tous : la sécurité repose uniquement sur le secret d'une clé.

Les algorithmes à clés secrètes ou symétriques sont des algorithmes où la clé de chiffrement peut être calculée à partir de la clé de déchiffrement et vice versa (les clés sont identiques dans la plupart des cas). L'émetteur et le destinataire doivent se mettre d'accord sur cette clé afin d'être utilisé avant tout envoi de message.

1. Les modes de chiffrement

Il y a deux modes de chiffrement : chiffrement par bloc (block cipher) et chiffrement en continu (stream cipher) ou chiffrement à flot.

a) Le chiffrement par bloc

La plupart des algorithmes sont des algorithmes de chiffrement par blocs. Un algorithme de chiffrement par bloc manipule des blocs de texte en clair et de textes chiffrés généralement de 64 bits. On cite quelques exemples d'algorithmes de chiffrement par blocs :

- **DES** (clé de 56 bits)
- **3DES** (trois clés distinctes 168 bits)
- **IDEA** (128 bits)
- **CAST-128** (128 bits)
- **Blowfish** (longueur de clés variable, jusqu'à 448 bits)
- **AES** (longueur de clés variable 128, 192, 256).

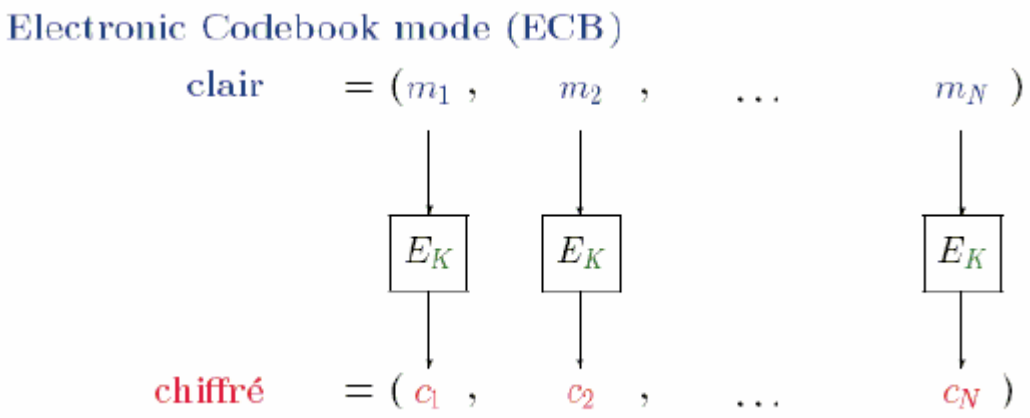
Les algorithmes de chiffrement par blocs peuvent être utilisés de différentes manières :

1. Electronic Code Book(ECB)

La méthode la plus évidente pour utiliser un algorithme de chiffrement par blocs est de chiffrer chaque bloc de texte en clair en un bloc de texte chiffré. On parlera donc d'un livre de codage électronique(ECB), car un même bloc de texte sera toujours chiffré en un même bloc de texte chiffré.

- C'est le mode opératoire le plus simple à utiliser. L'avantage est que chaque bloc de texte en clair est chiffré indépendamment.

- Le défaut du mode ECB est que si le cyptanalyste a le texte en clair et le texte chiffré de plusieurs messages, il peut commencer à construire un livre de codage sans connaître la clé.



2. Les modes de chiffrement avec chaînage de blocs (CBC, CFB, OFB)

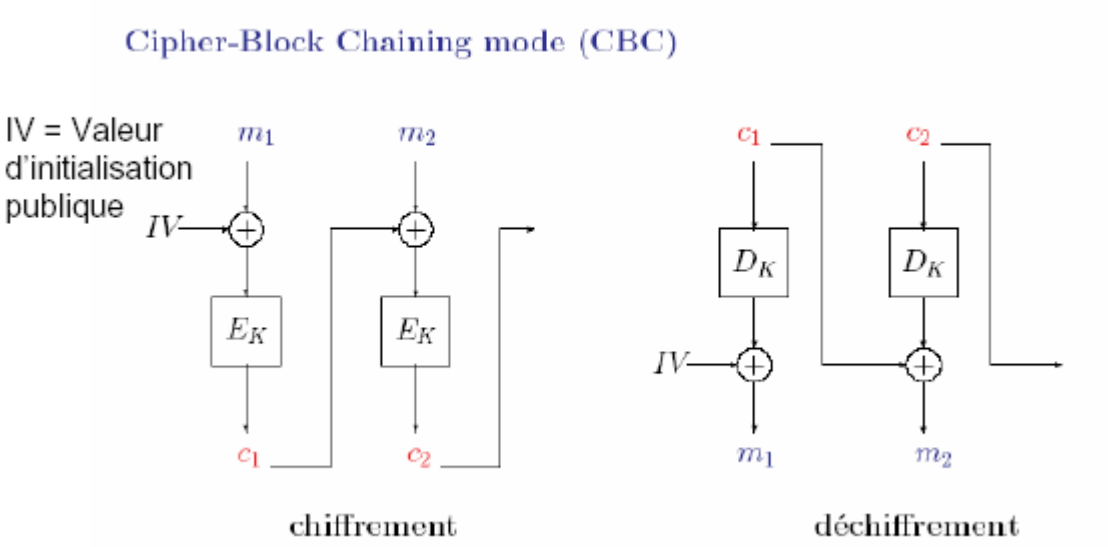
Le chaînage utilise une méthode de rétroaction car le résultat de chiffrement du bloc précédent est réutilisé comme entrée pour le chiffrement du bloc courant.

En mode de chiffrement par chaînage de blocs CBC (Cipher Bloc Chaining), voir figure, le texte en clair est combiné par un ou exclusif avec le bloc chiffré précédent avant d'être chiffré. Après qu'un bloc de texte en clair a été chiffré, le texte chiffré correspondant est aussi stocké dans un registre de rétroaction. Avant que le bloc suivant de texte en clair soit chiffré, il est combiné par un ou exclusif avec le registre de rétroaction pour devenir la nouvelle entrée de l'algorithme de chiffrement. Ce processus se répète jusqu'à la fin du message.

On peut écrire (M : message en clair ; C : le texte chiffré et E la fonction de chiffrement)

$$C_i = E_k(M_i \oplus C_{i-1})$$

On peut commencer le chiffrement par l'utilisation d'un bloc aléatoire simple dont on le laisse en clair, cela ne nuit pas à la sécurité, ce bloc s'appelle aussi IV (Initialisation Vector).



Ce code est performant, mais en cas d'erreur dans le texte chiffré, il propage l'erreur dans le reste du texte. De plus avec ce mode, le chiffrement ne peut pas commencer avant qu'un bloc de données complet ait été reçu.

Le OFB et le CFB utilisent le bloc précédent pour modifier la clé chiffrant le bloc en cours.

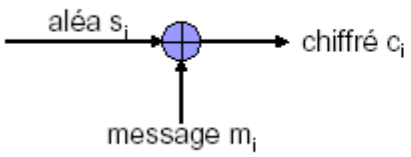
- **OFB (**Output FeedBack Mode **) :** on engendre une séquence de clés z_1,z_2,z_3, … par les formules z_0=IV, $z_i=E_k(z_{i-1})$, i>=1, et le chiffrement se fait par un ou exclusif $y_i=m_i \oplus z_i$, i>=1.

- **CFB (**Cipher feedback Mode**)** : fonctionne de la même manière que OFB sauf que la clé $z_i = E_k(y_{i-1})$, i>=1, est produite en chiffrant le bloc de texte chiffré précédent, y_0=IV et $y_i=m_i \oplus z_i$.

Leur niveau de sécurité est de l'ordre du mode CBC. Ces deux modes sont surtout utilisés pour les téléphones hautement sécurisés. Ils utilisent une très longue clé combinée avec un flot de données pour générer un texte chiffré.

b) chiffrement continu (ou chiffrement à flot)

Les algorithmes de chiffrement en continu convertissent le texte en clair en texte chiffré un bit à la fois. Un générateur aléatoire engendre, un flux de bits S (0 et 1) pour être combiné par un ou exclusif avec le flux de bit du texte en clair. Un exemple d'algorithme est RC4 (longueur de clés variable, généralement 128 bits)



$$m = m_0\ m_1\ m_2\ m_3 \ldots$$
$$\oplus\ s = s_0\ s_1\ s_2\ s_3 \ldots$$
$$= c = c_0\ c_1\ c_2\ c_3 \ldots$$

La sécurité de ces algorithmes repose sur les qualités du générateur aléatoire.

Pourquoi l'utilisation des chiffrements à flot ?

- Utilisation pour le software : chiffrement très rapide.
- Utilisation en hardware avec des ressources restreintes.
- Ne propage pas les erreurs (souvent utilisé en téléphonie mobile) (à la différence du chiffrement par blocs)

c) comparaison entre les deux modes de chiffrement

- Les systèmes de chiffrement continu sont plus faciles à cryptanalyser par rapport à ceux par bloc ; d'un autre coté les systèmes de chiffrement en continu sont plus faciles à analyser mathématiquement.
- Avec le chiffrement par bloc, une seule erreur corrompra au moins un bloc de données, tandis que le chiffrement continu est réalisé de telle façon qu'un bit de texte chiffré corrompu engendre un seul bit de texte chiffré corrompu.
- Un avantage du chiffrement par blocs est sa capacité d'opérer (c'est-à-dire chiffrer un grand nombre de blocs) en général plus rapidement que le chiffrement continu.

2. Exemple de chiffrement symétrique : DES

On l'appelle parfois DEA (Data Encryption Algorithm). C'est en fait une modification du système LUCIFER préparé par IBM. Il est en principe réévalué tous les cinq ans et encore utilisé sous différentes versions. Il était prévu qu'il s'arrête en 1998, et justement en 2000 est apparue une nouvelle norme AES (Advanced Encryption Standard). Ce dernier standard n'est pas encore généralisé et le DES est encore considéré comme sûr pour quelques années.

1. Principe du code DES

La cryptographie traditionnelle mettait en œuvre des algorithmes plus ou moins simples et des clés longues pour en assurer la sécurité. La tendance moderne s'est en quelque sorte inversée pour concevoir des algorithmes plus sophistiqués avec des clés courtes, le chiffrement se faisant souvent par blocs. Ainsi les premières variantes du DES transforment un message clair de 64 bits en un message chiffré de 64 bits (8 octets) avec une clé secrète de 56 bits. L'idée de base du code est de construire la fonction de chiffrement comme une combinaison de substitutions et de

permutations. On a donc un système itéré où *N* est le nombre d'itérations (facteurs du produit : on dit encore tours ou rondes). Dans le cas du DES, le nombre d'itérations est *N* =16.
Ces deux idées de chiffrement par blocs avec une clé courte et un système itéré se retrouve déjà dans LUCIFER. La troisième idée est que la fonction de chiffrement soit « facile » à inverser et il est proposé qu'elle soit elle-même sous forme d'un produit de transformations dont l'une au moins est linéaire et de substitutions). La facilité d'inversion est aussi réalisée par le fait que les 16 tours sont toutes identiques, mais paramétrées par diverses fonctions de la clé. L'algorithme est lui-même conçu de telle sorte à pouvoir effectuer le déchiffrement avec la même clé. Par contre, chacune des 16 itérations est contrôlée par des sous-clés dérivées de la clé maître *k*. Les étapes sont simplement exécutées dans l'ordre inverse.
En fait, le principe qui caractérise les systèmes actuels est l'utilisation du principe de *confusion/diffusion*. La confusion consiste à combiner des opérations linéaires et non linéaires afin de cacher les relations entre le clair, le chiffré et la clé. La diffusion quant à elle consiste à utiliser des transpositions pour faire dépendre chaque bit du message chiffré de tous les bits du message clair et de la clé.

Clé de 56 bits

Principe général du fonctionnement du code DES

Au cœur du DES. Dans le cas général, l'ensemble des messages est $\mathcal{M}=\mathcal{C}=\{0,1\}^{2n}$ (pour le DES dans la version décrite ici, $2n=64$). Chaque message est séparé en 2 blocs de 32 bits : $x = L(R)$ (noté parfois $x = LR$ pour indiquer que c'est la concaténation des listes L et R) : $L, R \in \{0,1\}^n$, L représente les 32 bits de « gauche » de x et R les 32 bits de droite (L pour left et R pou-r Right). Soit l'application $F : \{0,1\}^{m+n} \to \{0,1\}^n$ définie plus loin et appelée fonction ronde ou fonction itération.
La fonction DES est donc constituée de d=16 tours contrôlés chacun par les « sous-clés » $(k_1, k_2, ..., k_n)$, $k_i \in \{0,1\}^n$, ici $m = 48$, dérivés simplement de la clé maître k par des rotations successives et des extractions de bits avec pour des raisons techniques, l'intervention initiale et terminale d'une permutation π sur les bits :

$$L_0 \| R_0 = \pi(x)$$

$$L_i = R_{i-1},\ i = 1,2,...,16$$

(1)

$$R_i = L_{i-1} \oplus F(k_i, R_{i-1}),\ i = 1,2,...,16$$

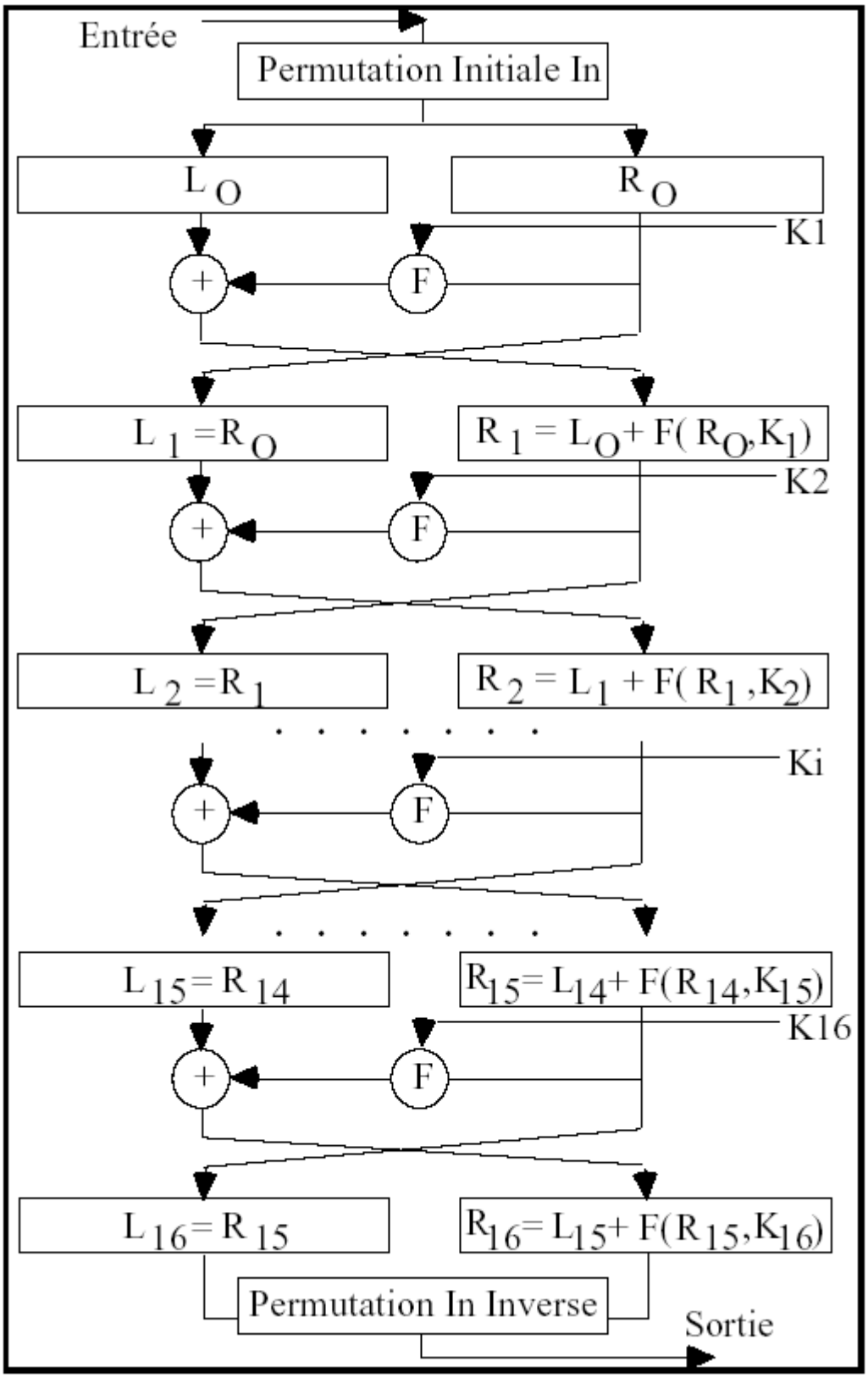
(2)

$$y = \pi^{-1}(R_{16}, L_{16})$$

Le symbole $\oplus$ représente l'addition modulo 2 des vecteurs binaires à n dimensions. Le message chiffré est obtenu par $d = 16$ applications des relations récurrentes (1) et (2).

L'introduction de la transformation $T : (L_{i-1}, R_{i-1}) \to (L_i, R_i)$ est motivée par le fait qu'elle peut-être inversée sans faire appel à la réciproque de F en utilisant les relations

$$R_{i-1} = L_i,\ i = 1,2,...,16$$
$$L_{i-1} = R_i \oplus F(k_i, L_i),\ i = 1,2,...,16$$


Entrée
Permutation Initiale In
L O
R O
K1
+
F
L 1 =R O
R 1 = L O+ F(R O,K 1)
K2
+
F
L 2 =R 1
R 2 = L 1 + F(R 1 ,K 2)
Ki
+
F
L 15= R 14
R15= L14+ F(R 14,K 15)
K16
+
F
L 16= R 15
R16= L15+ F(R 15,K 16)
Permutation In Inverse
Sortie

La fonction de déchiffrement est donc presque la même que la fonction de chiffrement. La transformation *T* est représentée sur la figure suivante.

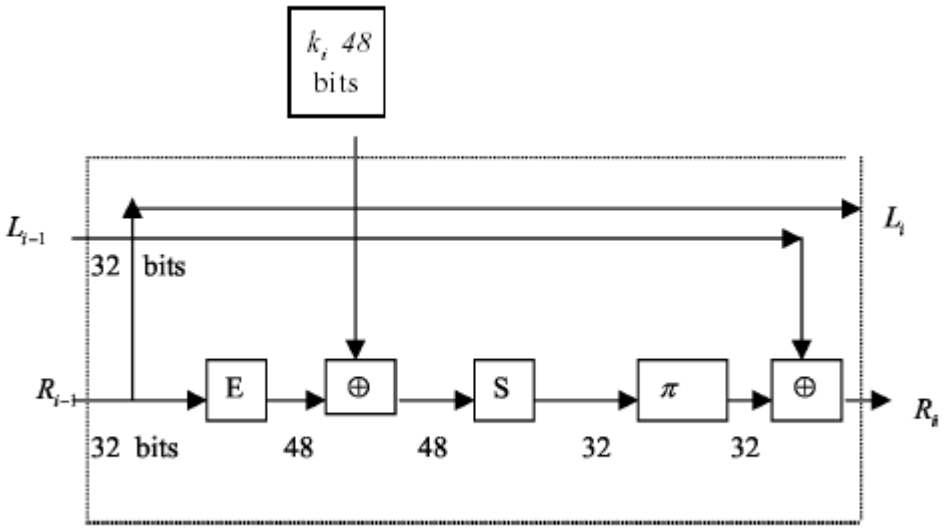


Figure 2. Un tour du DES

La Fonction ronde $F(k,R)=\pi(S(E(R))\oplus k_i)$est de structure assez complexe conçue comme un produit d'applications linéaires et de substitutions. Les systèmes utilisant des fonctions F bijection non linéaire sont dits systèmes de chiffrement de Feistel.

(i) $E:Z_2^{32}\to Z_2^{48}$ est une application linéaire qui transforme l'argument R de 32 bits en un mot $E(R)$ de 48 bits à l'aide de la fonction d'expansion E . $E(R)$ est composé de tous les bits de R dans un certain ordre, 16 d'entre eux sont dédoublés et apparaissent deux fois.
(ii) Le résultat du calcul de $B=E(R)\oplus k$ est séparé en 8 chaînes de 6 bits chacune : $B=B_1\|B_2\|...\|B_n$

(iii) $S:\{0,1\}^{48}\to\{\{0,1\}^{32}\}$ est une transformation non linéaire qui découpe $\{0,1\}^{48}$ en 8 blocs de 6 bits sur chacun desquels agit une transformation $S_i:\{0,1\}^6\to\{0,1\}^4$, $i=1,2,...,8.$ (qui peut être vu comme un tableau 4×16). Ces transformations sont aussi appelées S - boîtes ou boîtes de substitution. Pour chacune des sous-chaînes B_i de la chaîne B calculé à l'étape précédente , on calcule une nouvelle chaîne de 4 bits $C_j=S_j(B_j), j=1,2,3,4.$ Ainsi, chaque chaîne B_j ,de 6 bits, est modifiée par la boîte S_j de la manière suivante : (a) Soit une chaîne de 6 bits $B=b_1b_2b_3b_4b_5b_6$. on calcule les entiers $I=(b_1b_6)_2$, (représentation binaire de la ligne I de S_j) et $J=(b_2b_3b_4b_5)$ (représentation binaire de la colonne j de S_j), $0\le I,J\le 15.$ (b) La chaîne $C_j=S_j(B_j)=S_j(I,J),1\le j\le 8$ peut être vue comme la représentation binaire sur 4 bits du coefficient $S_j(I,J)$.

(iv) La chaîne $C=C_1\|C_2\|C_3\|C_4\|C_5\|C_6\|C_7\|C_8$ (de longueur 32) des 8 blocs concaténés est réordonnée selon une permutation fixée π et le résultat $\pi(C)=F(k,R)$ définit la fonction ronde. La fonction ronde F est ainsi une substitution (définie par les boîtes S) suivie d'une permutation et représente un crypto système produit au sens du chapitre précédent.

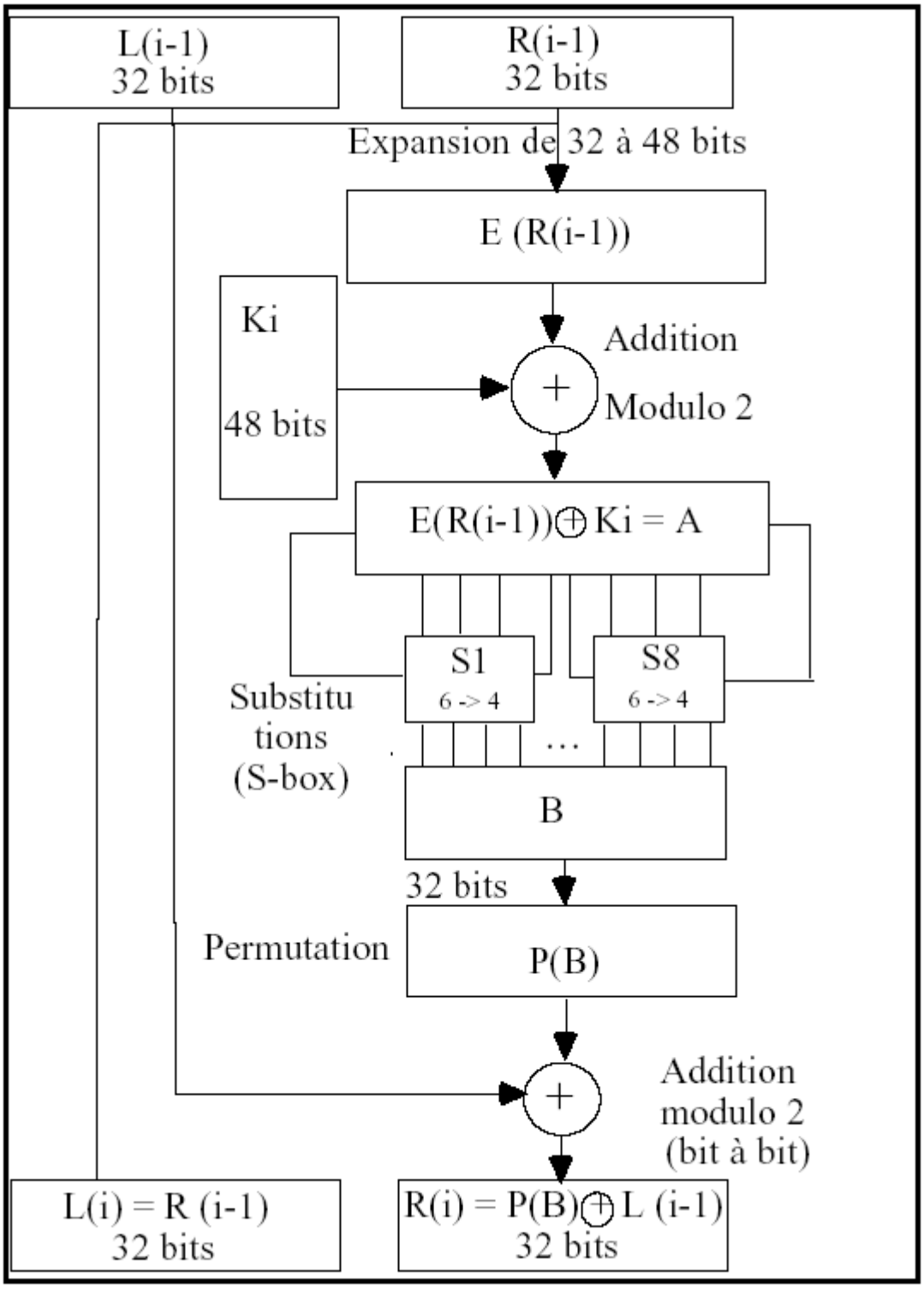
La fonction d'expansion E ou permutation expansive est définie à partir de la table

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Elle transforme un mot de 32 bits R en un mot de 48 bits) $E(R)$: la table indique les bits qui sont dédoublés : 32, 1, 4, 5, 8, 9, 12, 13, 16, 17, 20, 21, 24, 25, 28, 29.

Voici la boîte S_1, la permutation π, et la permutation inverse π^{-1}

Boîte S_1															
14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13


L(i-1)
32 bits
R(i-1)
32 bits
Expansion de 32 à 48 bits
E (R(i-1))
Ki
48 bits
Addition
Modulo 2
E(R(i-1))⊕Ki = A
S1
6 -> 4
S8
6 -> 4
...
Substitu
tions
(S-box)
B
32 bits
Permutation
P(B)
Addition
modulo 2
(bit à bit)
L(i) = R (i-1)
32 bits
R(i) = P(B)⊕L (i-1)
32 bits

Permutation π									Permutation inverse π^{-1}							
58	50	42	34	26	18	10	12		40	8	48	16	56	24	64	32
60	52	44	36	28	20	12	4		39	7	47	15	55	23	63	31
62	54	46	38	30	22	14	6		38	5	46	14	54	22	62	30
64	56	48	40	32	24	16	8		37	5	45	13	53	21	61	29
57	49	41	33	25	17	9	1		36	4	44	12	52	20	60	28
59	51	43	35	27	19	11	3		35	3	43	11	51	19	59	27
61	53	45	37	29	21	13	5		34	2	42	10	50	18	58	26
63	55	47	39	31	23	15	7		33	1	41	9	49	17	57	25

Génération des sous-clés. Le générateur est en fait un procédé de diversification des clés au cours des 16 tours. En fait DES utilise une clé k de 56 bits stockée sur 64 bits, dont 8 sont des bits de parité ou de détection d'erreur (un bit sur 8). Les bits en position 8,16,…,64 sont tels que chaque octet contient un nombre impair de 1. Ainsi, une seule erreur peut être détectée dans chaque groupe de 8 bits. On peut précalculer les 16 sous-clés $k_1, k_2, ..., k_{16}$ de 48 bits chacunes qui dépendent des 56 bits de la clé k. La création de chacune des 16 sous-clés (dépendant toutes des 56 bits de la clé initiale) utilise des fonctions de sélection et des permutations. (a) Enlever les bits de parité des 64 bits de k, puis réordonner les autres suivants une permutation Π_1, $\Pi_1(k) = C_0 D_0$ où C_0 est composé des 28 premiers bits de $\Pi_1(k)$ et D_0 des 28 restants. (b) Calculer $C_i = LS_i(C_{i-1})$; $D_i = LS_i(D_{i-1}), 1 \le i \le 16$ et $k_i = \Pi_2(C_i \| D_i)$, où LS_i est une rotation circulaire (vers la gauche) d'une ou deux positions, suivant la valeur de i : on décale d'une position si $i = 1,2,9$ ou 16 et on décale de deux positions sinon. Π_2 est une autre permutation des bits de chaîne.

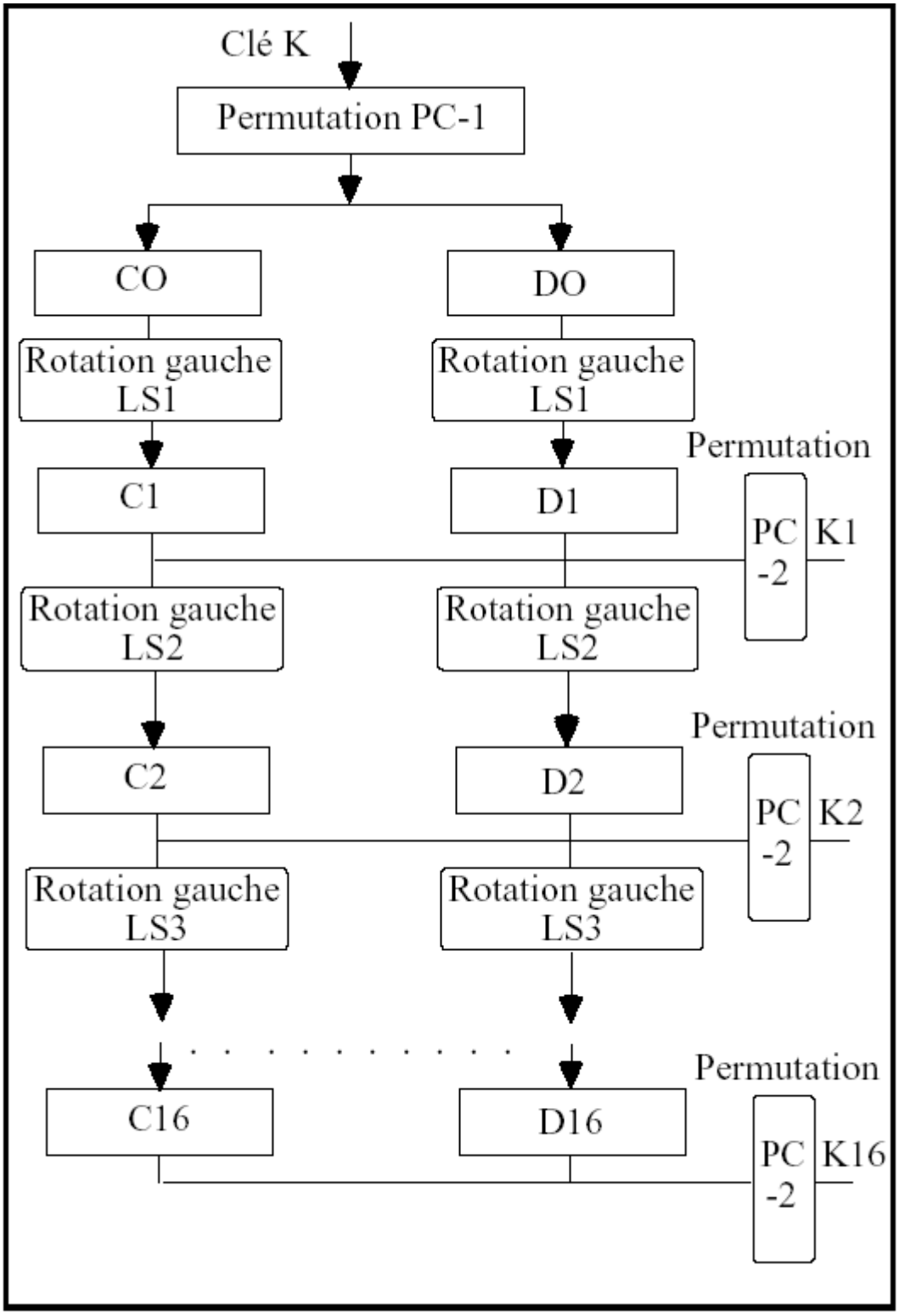


Figure : Génération des sous clés dans DES

Les permutations π1 et π2 sont les suivantes :

Permutation Π_1						
57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

Permutation Π_2					
14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	43
46	42	50	36	29	32

La clé initiale K est sur 64 bits.

- La permutation Π_1 enlève les bits de parité et opère sur les 56 bits restants.
- On divise le résultat en deux moitiés C_0 et D_0 de 28 bits.
- On génère une suite Ci, Di en opérant des décalages à gauche successifs:

✓ $C_i = LS_i (C_{i-1})$

✓ $D_i = LS_i (D_{i-1})$

- Pour obtenir la clé K_i on regroupe C_i et D_i et l'on opère sur les 56 bits une permutation Π_2

✓ $K_i = \Pi_2 (C_i D_i)$

Avantages et faiblesses du DES

(a) avantages

• Algorithme facile à implémenter (par logiciel et surtout matériel) ; vitesse de chiffrement rapide.

• Le déchiffrement est réalisé par le même algorithme que le chiffrement (modifier le plan de génération des clés). Ainsi, le même processeur, la même carte ou le même logiciel sert à chiffrer et à déchiffrer
• Utilise le principe de confusion/diffusion.
• Effet d'avalanche : chaque petit changement dans le texte clair entraîne un changement dans la moitié du texte chiffré.

(b) Inconvénients et faiblesses.
• Longueur de la clé : pourquoi une longueur aussi faible a-t-elle été choisie alors que la proposition initiale d'IBM préconisait 64 bits (on dit même 128)? Cela s'explique peut-être par le souci d'utiliser les bits de parité ? Cette petite clé semble constituer la seule faiblesse de DES. La faisabilité de la recherche exhaustive sur des clés de 56 bits a été démontrée mais au moyen de grandes quantités de calcul et moyennant de gros budgets. Les autres types d'attaque sont plutôt théoriques et restent encore inefficaces.
• Dans tous les cas, la sécurité du DES semble reposer sur les S - Boîtes : il a été montré en effet que tout autre choix pour ces boîtes aurait conduit à un algorithme plus vulnérable face aux attaques différentielles et linéaires.

Modification du DES
En pratique pour renforcer la sécurité, on utilise parfois le DES à 2 ou trois clés.

- Le 2DES ou DES à deux clés utilise $y= f_{k2}(f_{k1}(x))$, une recherche exhaustive nécessite donc d'explorer $2^{562}= 2^{128}$ possibilités. En réalité, l'attaque par collision basée sur le problème des anniversaires est qui est une attaque à texte clair connu montre qu'il suffit de 265 tentatives.
- Le 3DES ou TDES utilise trois clés distinctes en gardant le mode « chiffre déchiffre chiffre » $y = f_{k3}(f_{k2}\text{-}1(f_{k1}(x)))$. La recherche exhaustive nécessite 2^{128} tentatives et 2^{46} blocs mémoires. Si k1=k2=k3, on obtient le DES simple (l'inconvénient est la lenteur).

3. L'algorithme AES alternative au DES

Description
• Comme pour le DES c'est une méthode de chiffrement par blocs de 128 bits et est disponible pour trois tailles de clés : 128, 192 ou 256.
• Chaque bloc est représenté par une matrice d'octets (« état ») ayant 4 lignes et N_b= 4, 6 ou 8 colonnes.
• Le chiffrement d'un bloc est réalisé en d=10, 12 ou 14 tours.
• Chaque tour comprend 4 transformations: (i) substitution non linéaire *S* sur chaque octet de l'état ; (ii) inversion dans le corps de Galois à 256 éléments $GF(2^8)$; (iii) transformation linéaire π dans $GF(2^8)$. (iv) décalage cyclique de chaque ligne ; (v) transformation de chaque colonne par multiplication par un polynôme à coefficients dans $GF(2^8)$; (vi) *XOR* (ou exclusif) avec une sous-clé de tour.
• Ainsi chaque tour obéit au principe de confusion/diffusion de Shanon.

• La sous-clé de tour est dérivée de la clé secrète par : (i)expansion de la clé secrète ; (ii) expansion de la clé de tour.
• Plus précisément, la sous-clé N°i (utilisée au ième tour) est déduite de la sous-clé N°i-1.

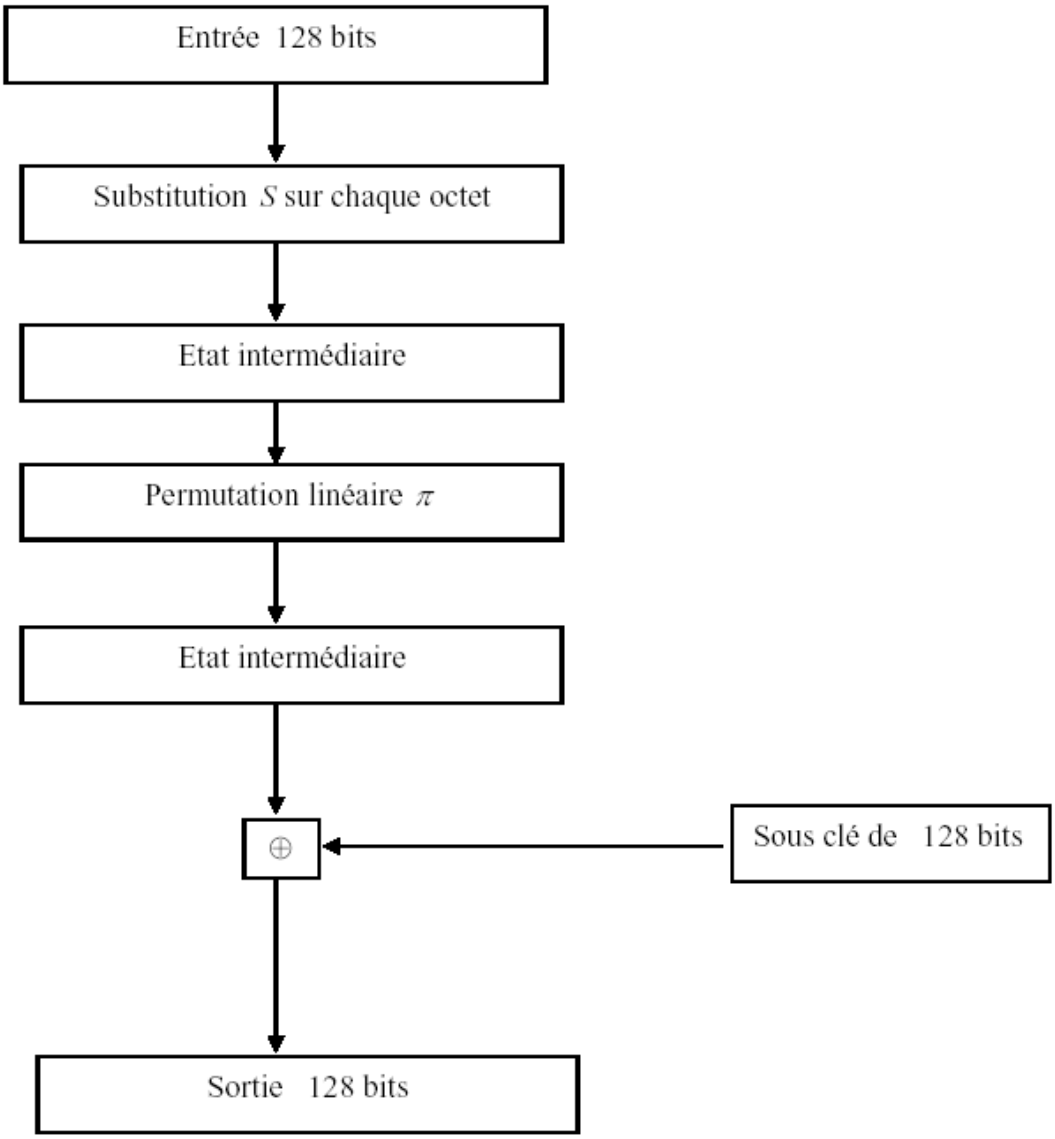


Figure: Une itération de l'AES

Avantages de l'AES

• Très rapide pour les processeurs 8 bits utilisés sur les cartes à puce et pour les implémentations logicielles. Ordre de grandeur : débit de chiffrement de 70 Mbits/s pour une Sous clé de 128 bits implémentation en C++ sur un Pentium à 200 MHz. C'est dû en partie au fait que l'algorithme utilise des opérations simples sur les octets
• Conception relativement simple.
• Répond aux normes de sécurité requises : semble plus sûr que le 2 ou 3DES.

4. IDEA (International Data Encryption Algorithm)

La première version du système IDEA a été développée à Zurich par James L.Massey et Xujeia Lai, et publiée en 1990. Il s'appelait PES «Proposed Encryption Standard ». L'année suivante après que Biham et Shamir eurent présenté la cryptanalyse différentielle, les auteurs ont renforcé leur système pour résister à cette attaque. Il fut rebaptisé IPES puis finalement IDEA. Il s'agit de l'un des meilleurs algorithmes de chiffrement, il est breveté aux U.S.A, au Japon et quelques pays de l'Europe.

Description

C'est un algorithme de chiffrement par blocs de 64 bits. On utilise souvent des blocs de cette grandeur. Car il faut que le bloc soit assez grand pour être pratique et suffisamment petit pour que la cryptanalyse soit compliquée. Sa clé est de 128 bits, donc le double du DES. Il utilise également la combinaison de diffusion et de confusion comme DES. La diffusion n'est pas faite par des permutations dans le cas de IDEA.

IDEA est fondé sur le concept du «mixage d'opérations depuis différents groupes algébriques ». Les trois opérations élémentaires de l'algorithme sont le OU exclusif, l'addition modulo 2^{16} et la multiplication modulo 2^{16} +1. Il n'y a pas d'autres opérations. Les multiplications sont coûteuses en temps ce qui fait que IDEA est plus lent que DES. DES peut chiffrer 40600bloc/s sur un 486 cadencé a 33Mhz. IDEA n'en chiffre que 13750/s ce qui donne une différence de 13750 blocs = 110KO/s.

Fonctionnement

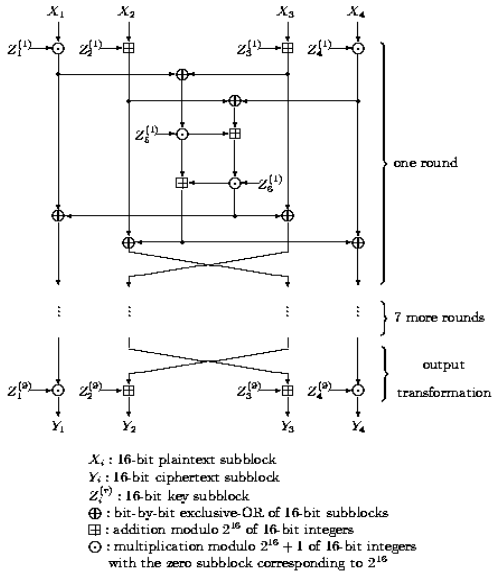
Le bloc de données de 64 bits est divisé en 04 sous blocs de 16 bits et la clé est divisée elle aussi en six sous clés de 16 bits. Les 04 sous blocs deviennent les entrées d'une première ronde. Dans une ronde les 04 sous blocs sont combinés par XOR, additionnés, et multipliés entre eux avec les 6 sous blocs de 16 bits dérivés de la clé(6*16 = 96 bits). A la sortie de la ronde, on retrouve les 04 sous blocs transformés.

Dans chaque ronde on effectue les opérations suivantes :

- ✓ Une multiplication modulo 2^{16} +1 entre X1 et Z1 noté A1 et X4 avec Z4 noté A2.
- ✓ Une addition modulo 2^{16} entre X2 avec Z2 noté A3 et X3 avec Z3 noté A4.
- ✓ Un XOR entre A1 et A3 résultat A5.
- ✓ Un XOR entre A2 et A4 résultat A6.
- ✓ Une multiplication entre A5 et Z5 résultat noté A7.
- ✓ Une addition entre A7 et A6 résultat A8.
- ✓ Une multiplication entre Z6 et A8 le résultat noté A9.
- ✓ Une addition entre A9 et A7 noté A10.
- ✓ Un XOR entre A9 et A3 résultat noté A11.

- ✓ Un XOR entre A9 et A1 noté A12.
- ✓ Un XOR entre A10 et A2 résultat noté A13
- ✓ Un XOR entre A10 et A4 résultat noté A14

A la fin de la ronde on inverse les positions de A11 et A13 et on passe à la ronde suivante. Les sous clé proviennent de plusieurs découpages et rotations sur les bits de la clé.



X_i : 16-bit plaintext subblock
Y_i : 16-bit ciphertext subblock
$Z_i^{(r)}$: 16-bit key subblock
$\oplus$: bit-by-bit exclusive-OR of 16-bit subblocks
$\boxplus$: addition modulo 2^{16} of 16-bit integers
$\odot$: multiplication modulo $2^{16}+1$ of 16-bit integers with the zero subblock corresponding to 2^{16}

On répète 8 fois toutes ces opérations, durant les 08 rondes.

Figure : Fonctionnement d'IDEA.

Déchiffrement avec IDEA

En ce qui concerne le déchiffrement, il s'agit de modifier la clé de chiffrement par des calculs d'inverses, puis d'utiliser exactement la fonction de chiffrement, avec en entrée le bloc chiffré. On a en sortie le bloc déchiffré. Il est possible de chiffrer un bloc de texte clair en le déchiffrant avec la clé. Puis pour retrouver le texte clair on le chiffre pour obtenir le résultat déchiffré.

Sécurité d'IDEA

La longueur de la clé IDEA est de 128 bits. Une attaque exhaustive demande d'essayer les 2^{128} clés. Un P.C. 486 aura besoin de 7.8*1026 ans pour essayer toutes les combinaisons. En supposant qu'on puisse aller mille milliards de fois plus vite on mettra 780.000 milliards d'années pour accomplir cette tâche. Ceci démontre que toute attaque exhaustive est impossible.

IDEA est immunisé à la cryptanalyse différentielle après seulement 04 rondes. Il y en a 08. Comme la plupart des algorithmes, il existe une série de clés faibles. Une clé faible est une clé particulière qui fait que le chiffrement devient faible. On a découvert des clés faibles qui sont du type 0000 0000 0x00 0000 0000 000x xxxx x000, ou x est un nombre quelconque en hexadécimal. La chance de générer une clé faible est très faible 1 sur 296 = 79228162514264337593543950336.

Malheureusement le plus grand obstacle à ce que IDEA devienne un standard a été le fait que Ascom Systec détienne un brevet sur sa conception. Et de ce fait il n'est pas disponible gratuitement et il faut une autorisation spéciale pour pouvoir le lui acheter.